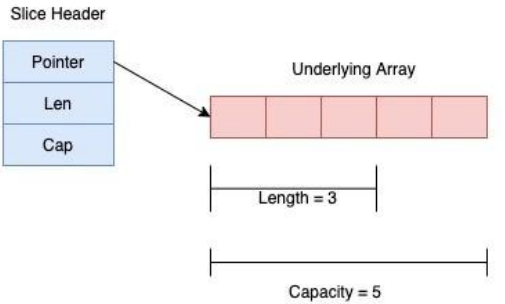
2.5.2 Составные типы. Слайсы

Слайс — это динамическая коллекция элементов, основанная на массиве



MERION

```go
package main
import "fmt"

func main() {

   var s1 []int
   s2 := []int{}
   s3 := make([]string, 0, 2)
   s4 := []int{3, 3, 5, 4, 5}

   fmt.Println(s1, s1 == nil, len(s1), cap(s1))
   fmt.Println(s2, s2 == nil, len(s2), cap(s2))
    fmt.Println(s3, s3 == nil, len(s3), cap(s3))
    fmt.Println(s4, s4 == nil, len(s4), cap(s4))
}
```

Создание слайса из массива

```
package main
import "fmt"


func main() {
    numbers := [5]int{1, 2, 3, 4, 5}
    newSlice := numbers[1:4]
    fmt.Println(newSlice)

    newSlice[1] = 11

    fmt.Println(newSlice)
    fmt.Println(numbers)
}
```

Добавление элементов в слайс

```
package main
import "fmt"

func main() {
    numbers := []int{1, 2, 3}
    numbers = append(numbers, 6)
    fmt.Println(numbers)
}
```

MERION

Копирование слайсов

```
package main
 import "fmt"
func main() {
    // copy func
    original := []int{1, 2, 3, 4, 5}
    copyOfOriginal := make([]int, len(original))
    copy(copyOfOriginal, original)

    // using append
    original := []int{1, 2, 3, 4, 5}
    copyOfOriginal := append([]int{}, original...)
}
```

MERION

Изменение элементов внутри функции влияет на оригинальный срез

```
package main
import "fmt"

func modifySlice(s []int) {
    s[0] = 999
}

func main() {
    original := []int{1, 2, 3}

    fmt.Println("До вызова:", original)
    modifySlice(original)
    fmt.Println("После вызова:", original)
}
```

MERION

Как избежать

```
package main
 import "fmt"

 func safeModify(s []int) []int {
 copySlice := make([]int, len(s))
    copy(copySlice, s)

    copySlice[0] = 999 return
    copySlice
 }

 func main() {
    original := []int{1, 2, 3}
    modified := safeModify(original)
    fmt.Println(original)
    fmt.Println(modified)
 }
```

Доступ к элементу за пределами диапазона длины запрещен

```
package main

 func main() {

    s := make([]int, 3, 6)
    s[1] = 7
    s[4] = 0
}
```

Неэффективная инициализация среза

```
var s []int
for i := 0; i < 1000; i++ {
  s = append(s, i)
}

s := make([]int, 0)
for i := 0; i < 1000; i++ {
  s = append(s, i)
}

s := make([]int, 0, 1000)
for i := 0; i < 1000; i++ {
   s = append(s, i)
}
```

MERION

Результаты

```
goos: darwin
goarch: arm64
pkg: temp
cpu: Apple M3 Pro
BenchmarkVarInit-11               609901              1960 ns/op           25208 B/op         12 allocs/op
BenchmarkMakeWithoutCap-11        618759              2056 ns/op           25208 B/op         12 allocs/op
BenchmarkMakeWithCap-11          2924220               414.2 ns/op             0 B/op          0 allocs/op
```

MERION

Пустой срез VS NIL-срез

nil-срез

```
package main
import "fmt"

func main() {
    var nilSlice []int

    fmt.Println(nilSlice == nil) // true
    fmt.Println("len: ", len(nilSlice))
    fmt.Println("cap: ", cap(nilSlice))
}
```

MERION

Пустой срез

```
package main
import "fmt"

 func main() {
  emptySlice := make([]int, 0) // Через make
  emptySlice2 := []int{} // Через литерал
  fmt.Println(emptySlice == nil) // false
  fmt.Println("len: ", len(emptySlice))
  fmt.Println("cap: ", cap(emptySlice))
}
```

Добавление элементов

```
package main
import "fmt"

 func main() {
    var nilSlice []int
    nilSlice = append(nilSlice, 42)

    emptySlice := make([]int, 0)
    emptySlice = append(emptySlice, 42)
 }
```

JSON сериализация

```
package main
import "fmt"

func main() {
    var nilSlice []int
    emptySlice := []int{}

    jsonNil, _ := json.Marshal(nilSlice)    // "null"
    jsonEmpty, _ := json.Marshal(emptySlice)    // "[]"

    fmt.Println(jsonNil)
    fmt.Println(jsonEmpty)
}
```

MERION

Когда что использовать?

Ситуация	Рекомендация
Нужно обозначить отсутствие данных	Используйте `nil`
Требуется пустая коллекция	Используйте `[]int{}` или `make([]int, 0)`
Проверка на наличие данных	Всегда проверяйте `len(s) == 0` , а не `s == nil`

Итоговая таблица

Критерий	Nil-срез	Пустой срез
Инициализация	`var s []int`	`s := make([]int, 0)`
Сравнение с `nil`	`true`	`false`
Базовый массив	Отсутствует	Присутствует (пустой)
JSON-сериализация → `null`	→ `[]`	

Побочные эффекты при использовании `append` в Go

Функция `append` в Go может вызывать неочевидные побочные эффекты из-за того, что срезы ссылаются на базовые массивы.

Рассмотрим примеры, где это приводит к неожиданному поведению.

Побочные эффекты при использовании `append`

```
package main
import "fmt"

 func main() {

   original := []int{1, 2, 3}
   slice := original[:2]

   slice = append(slice, 4)

   fmt.Println("original:", original)
   fmt.Println("slice:", slice)
}
```

Spread-оператор (распаковка слайсов)

Синтаксис `...` позволяет "распаковать" элементы слайса

```go
package main

import "fmt"

func main() {
    nums1 := []int{1, 2, 3}
    nums2 := []int{4, 5}

    // Объединение слайсов
    combined := append(nums1, nums2...)
    fmt.Println(combined) // [1 2 3 4 5]

    // Добавление нескольких элементов
    nums1 = append(nums1, 6, 7)
    nums1 = append(nums1, []int{8, 9}...)
    fmt.Println(nums1) // [1 2 3 6 7 8 9]
}
```

Использование в variadic-функциях

```
package main
import "fmt"

func printNumbers(numbers ...int) {
for _, n := range numbers {
       fmt.Println(n)
    }
}

func main() {
   nums := []int{1, 2, 3}
   printNumbers(nums...)
}
```

MERION

Важные особенности

1. Работает только со слайсами (не с массивами)
2. Можно использовать только в определенных контекстах:
    - аргументы variadic-функции
    - оператор append
3. Для массивов нужно сначала преобразовать в слайс:

```
arr := [3]int{1,2,3} printNumbers(arr[:]...) //
OK
```